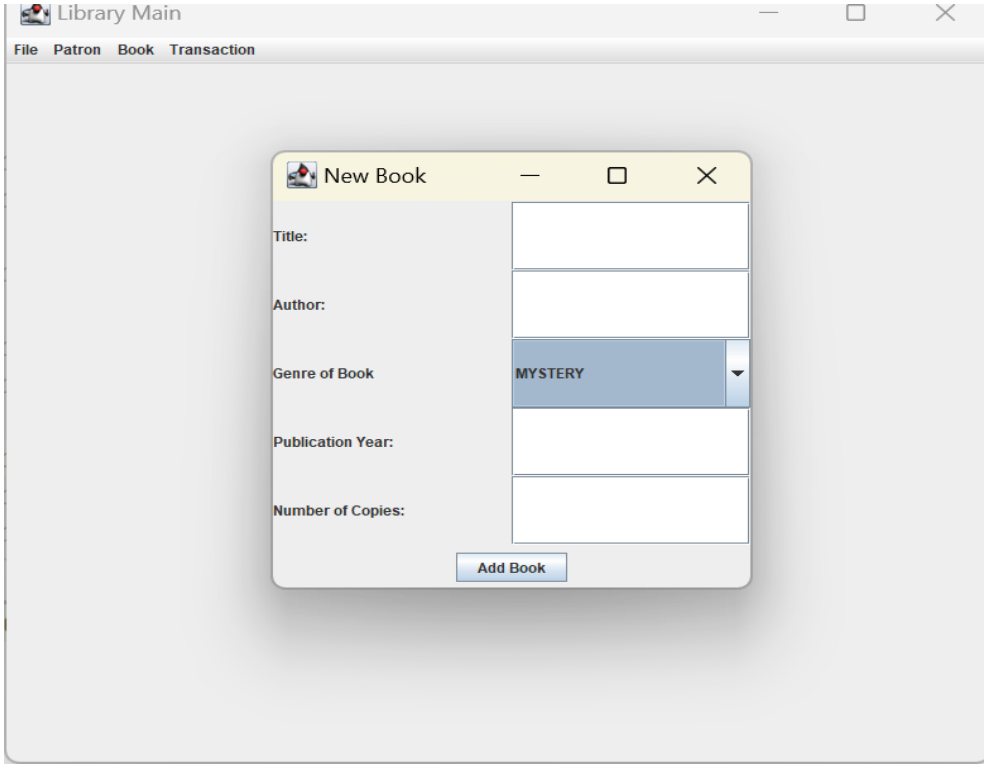
Lecture 12

Graphical User Interface

Introduction

- A graphical interface application is an application that uses a graphical interface to communicate with the user
  - Import and display data from graphical components



Java's support for GUI programming (1)

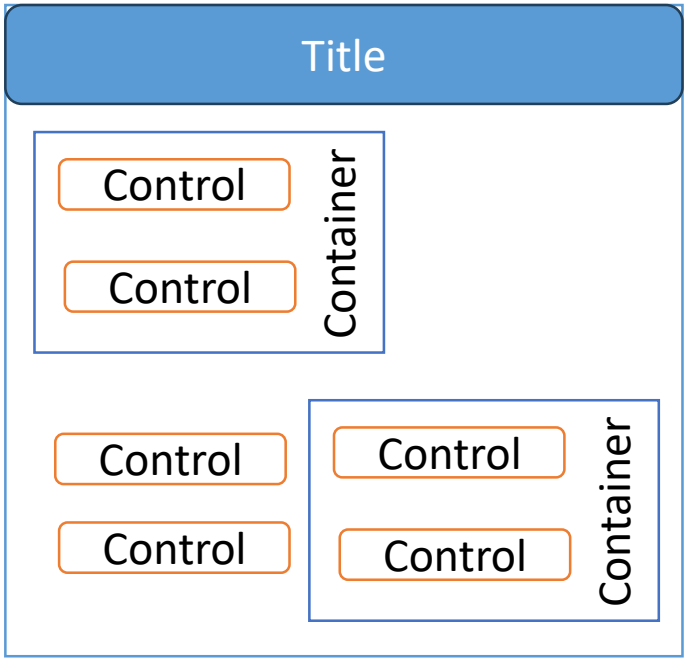
- Two APIs: AWT & Swing
- Abstract Window Toolkit (AWT):
  - (older) introduced since JDK 1.0
  - decouple display component specifications from their implementations
  - heavy-weight: uses a native peer class for each display component
  - the peer classes are fixed for given platform
  - has a modular interaction model (JDK 1.1)

Java's support for GUI programming (2)

- **Swing:**
  - introduced since JDK 1.2
    - improved from AWT
  - light-weight: Java's implementation of the display (no need for peer classes)
  - look & feel of the display components can be customized for a given platform
  - has enhanced display components
  - extends AWT’s interaction model from JDK 1.1

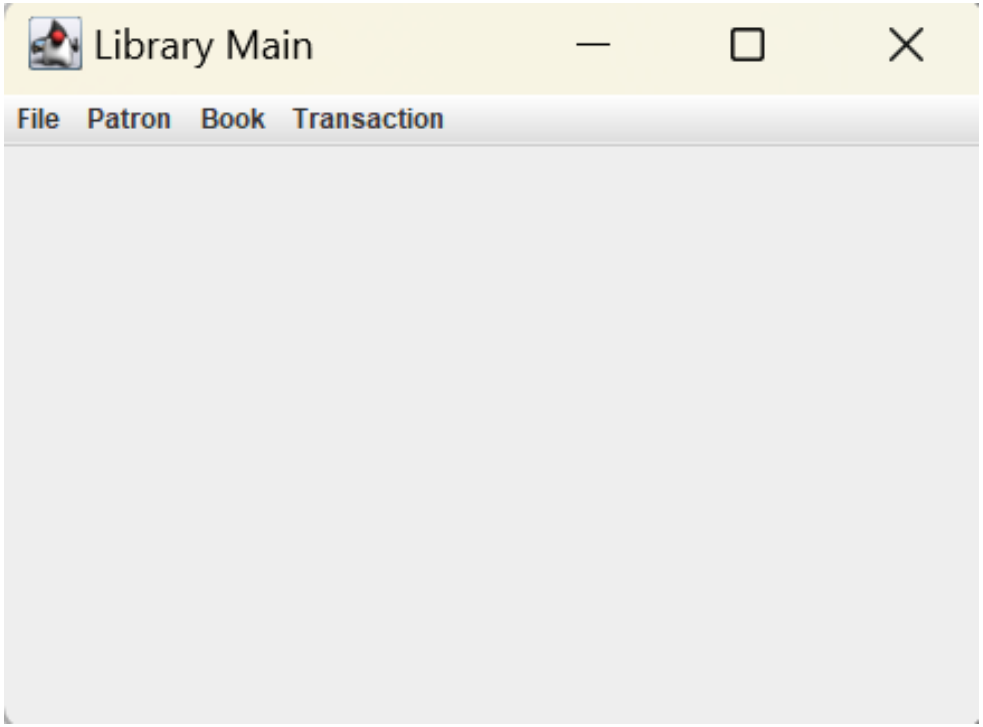
Basic components of GUI

- Window
  - Application window
- Container
  - Contains one or more controls
- Control
  - TextField, Radio, CheckBox, Buttons, Tables…
- Note:
  - An application can have many windows.
  - A container can contain multiple controls or other containers



Window

- In Swing, `JFrame` is used to create windows
  - `javax.swing.JFrame`
- Related components
  - Icon
  - Title
  - Minimize, maximize, and close buttons



Create a window object

- **Directly** using the `JFrame` class:

```
JFrame w = new JFrame();
JFrame w1 = new JFrame("First Window Class");
```

- **Indirectly** via a sub-class of `JFrame`:

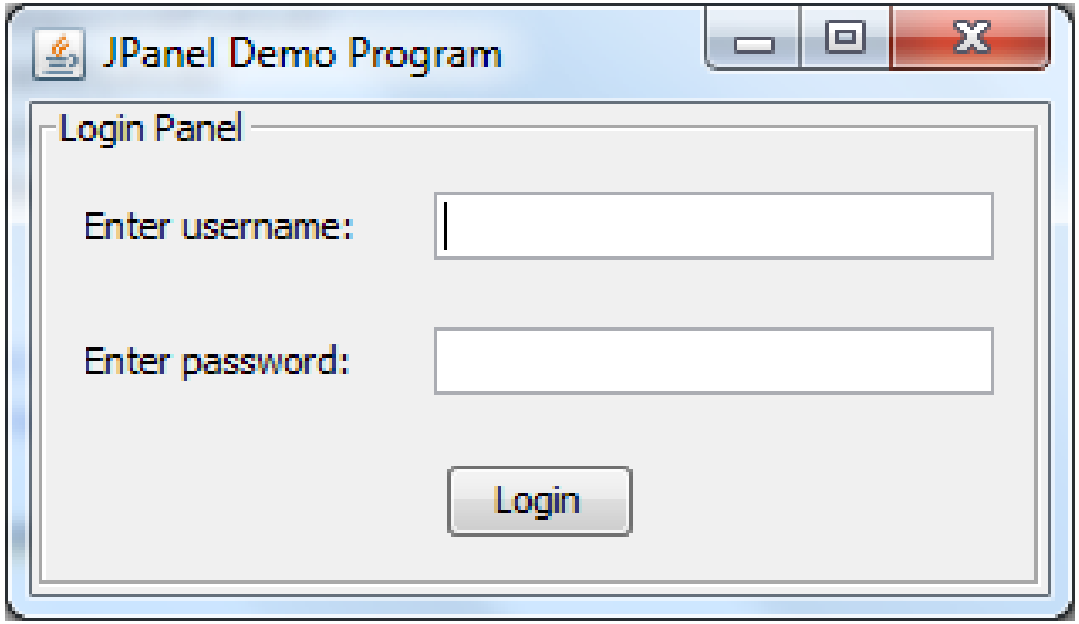
```
public class FirstWindow extends JFrame {...}
JFrame w = new FirstWindow();
```

Example

```
// Create a JFrame instance
JFrame frame = new JFrame();

// Set the title of the frame
frame.setTitle("Window Demo");

// Set the icon of the frame
ImageIcon icon = new ImageIcon("path/to/icon.png");
 // Replace with your icon path
frame.setIconImage(icon.getImage());

// Set the size of the frame
frame.setSize(400, 300);

// Set the default close operation
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
```

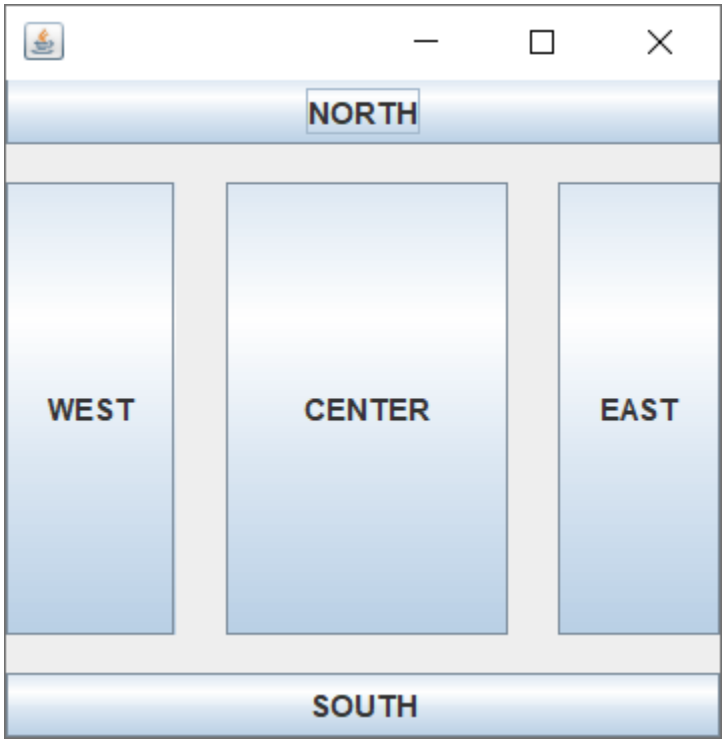
Container

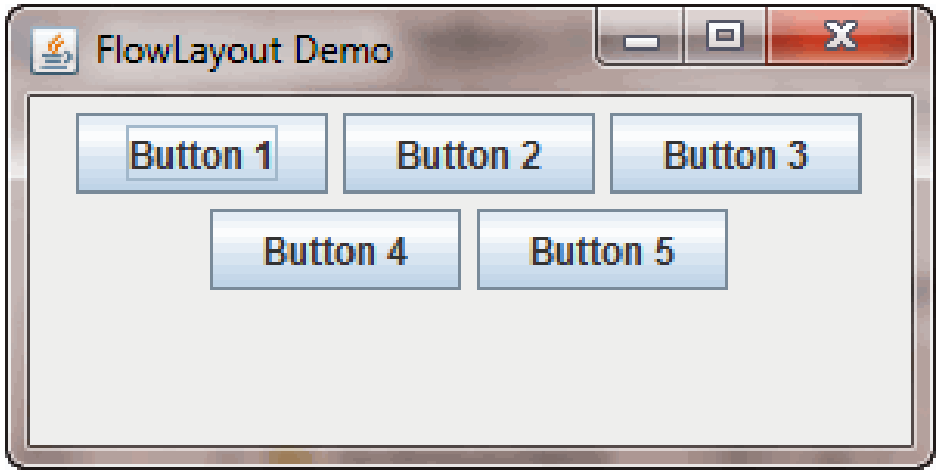
- In Swing there are many types of containers. In this lecture we will only learn about `JPanel`
- `JPanel` is a container used to group other controls or containers



Container Layouts

- A Layout (a.k.a. LayoutManager) dictates how components are arranged in a container.
- The default layout for `JFrame` is `BorderLayout`
- The default layout for `JPanel` is `FlowLayout`





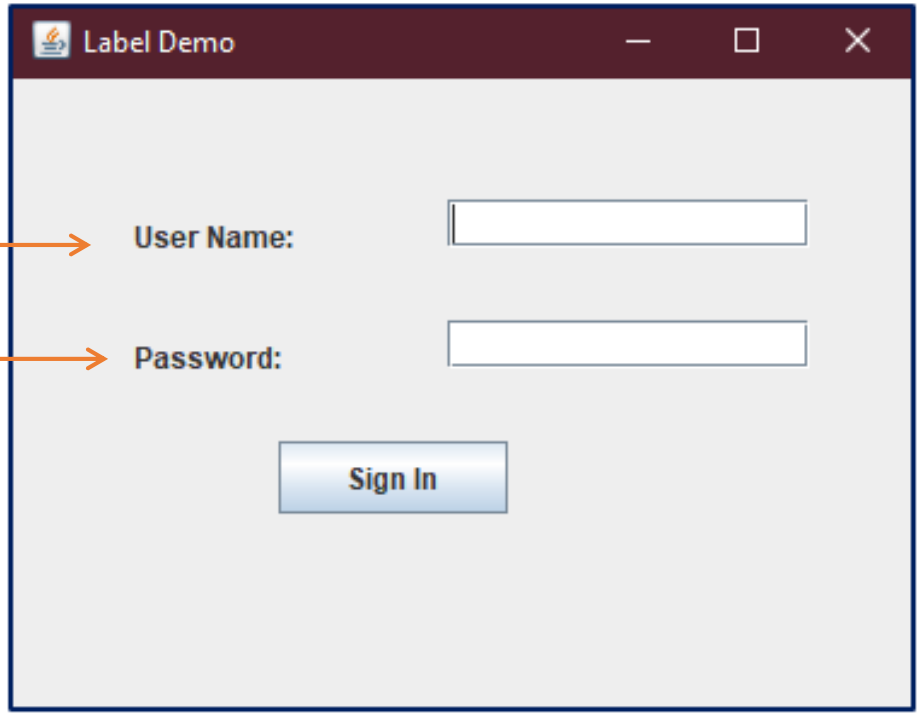
Controls

- Controls are interface elements used to receive data from the user or sender that presents data to the user
- Commonly used controls
  - `JLabel` – Text label
  - `JTextField` – Input field (one line)
  - `JButton` – Button
  - `JCheckBox` – Checkbox
  - `JRadioButton` – Radio button
  - `JComboBox` – Selection box (drop-down menu)
  - `JTable` – Table
  - `JOptionPane` – Dialog box (popup dialog window)
  - …

JLabel

- `JLabel` is the control used to create labels
- The most important attribute of `JLabel` is its text
- Label names you usually start with `lbl`

- `lblUserName`
- `lblPassword`

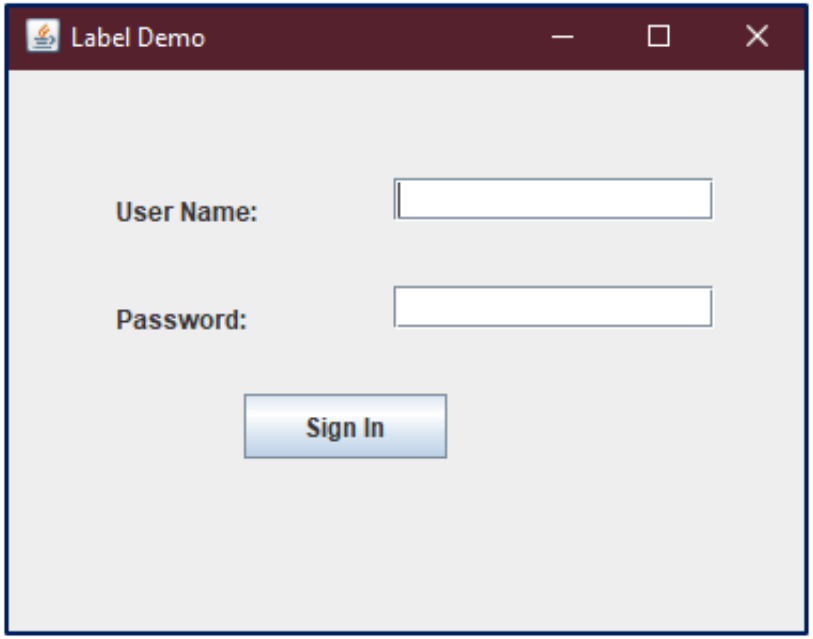


JLabel

```
JLabel lblInput; // declare a JLabel instance
lblInput = new JLabel("User Name:");
container.add(lblInput); // add the label to some container
lblInput.setText("Password");
String text = lblInput.getText(); // retrieve the text
```

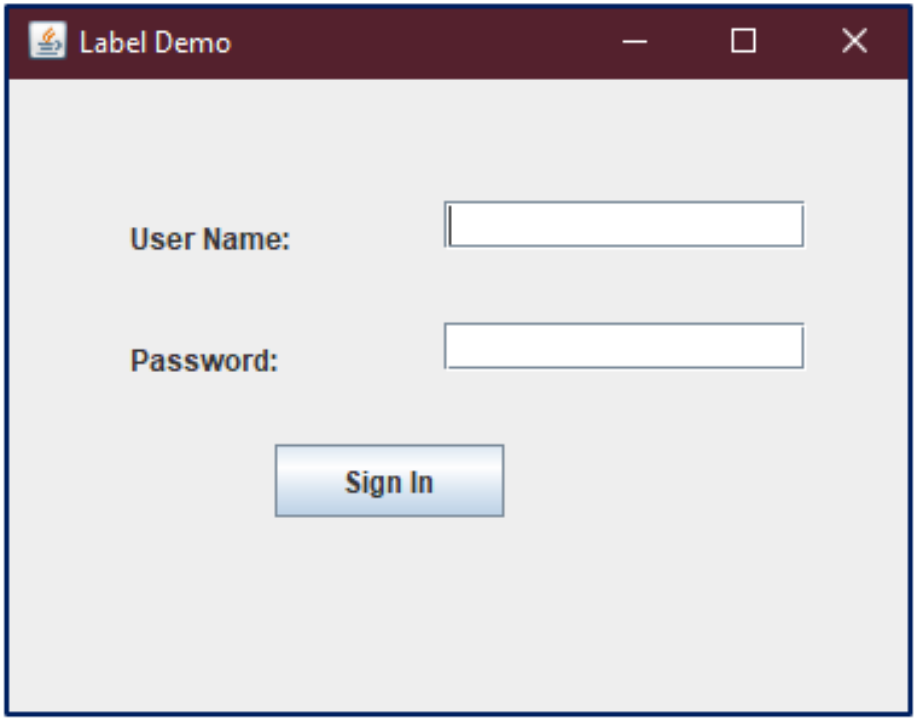
`JTextField`, `JTextArea` and `JPasswordField`

- `JTextField` is used to create input boxes
  - Conventionally, name of a text field should start with `txt`, for example:
  - ➢ `txtUsername`
  - ➢ `txtPassword`
- `JTextArea` is only different from `JTextField` in that it allows it to contain multi-line text
- `JPasswordField` does not allow the actual content to be seen



JTextField, JTextArea and JPasswordField

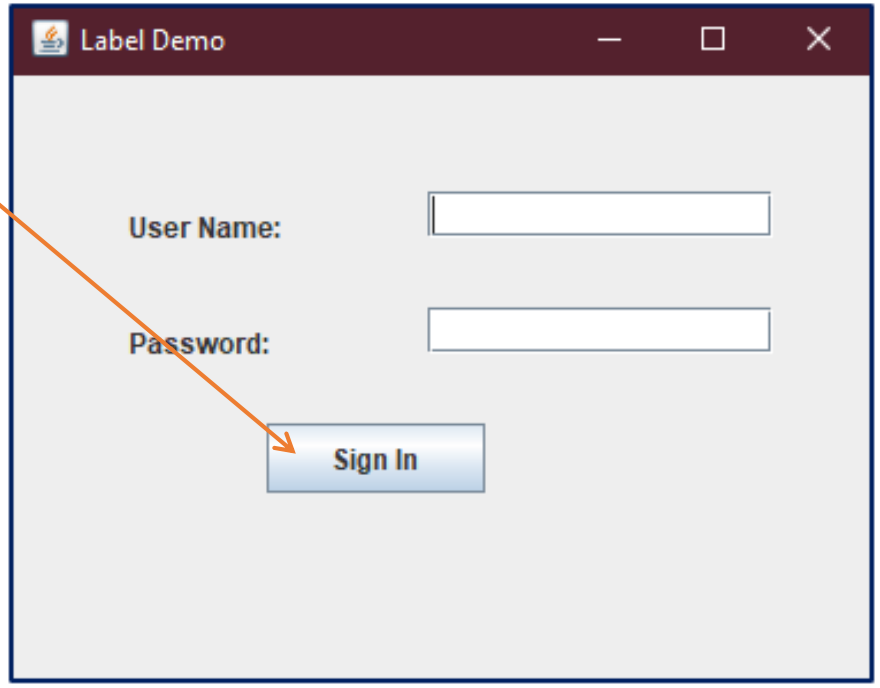
```
jPanel.add(new JLabel("User Name:"));
JTextField txtUsername = new JTextField();
jPanel.add(txtUsername);

jPanel.add(new JLabel("Password:"));
JTextField txtPassword = new JPasswordField();
jPanel.add(txtPassword);
```



JButton

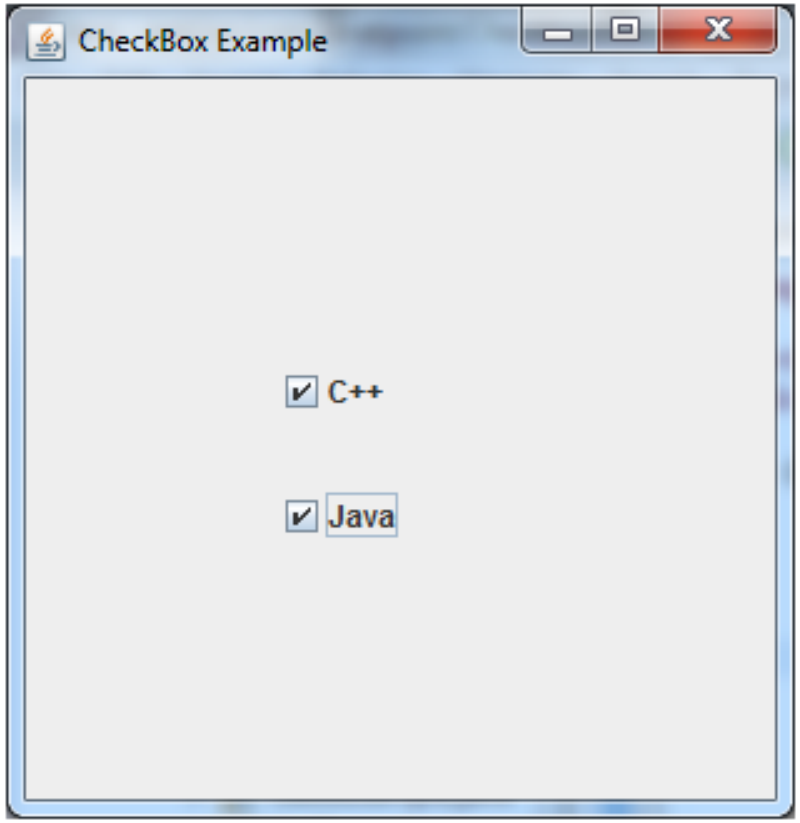
- `JButton` is used to create push buttons
  - Text: button label
  - By convention, name of the button should start with `btn`
  - For example: `btnLogin`

```
JButton btnLogin =
        new JButton("Sign In");
jPanel.add(btnLogin);
```



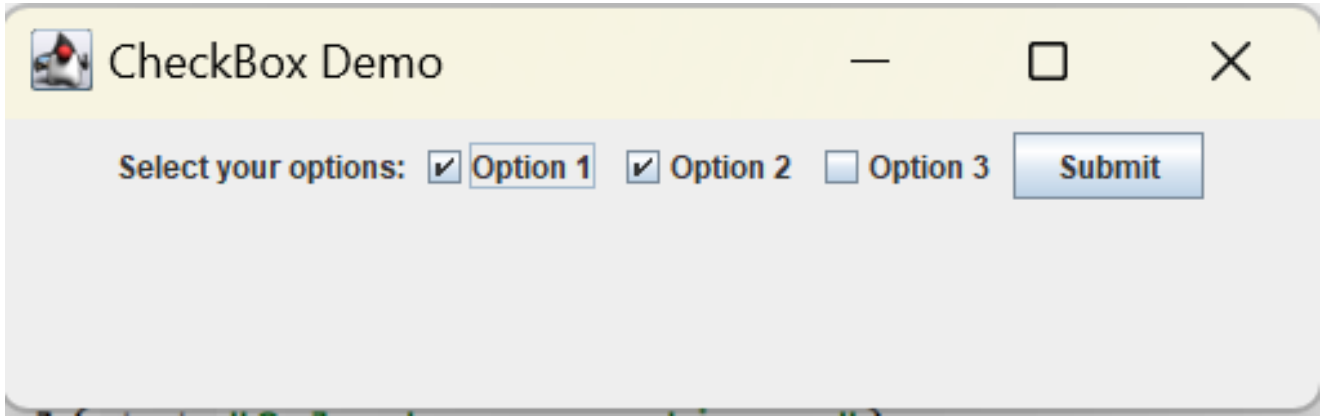
JCheckBox

- `JCheckBox` is used to create checkboxes.
- Commonly used attributes:
  - Text: Attached label
  - Selected: Status (checked or unchecked)
- Conventionally, name should start with `chk`



JCheckBox

```
JPanel jPanel = new JPanel();
JLabel lblSelect = new JLabel("Select your options:");
jPanel.add(lblSelect);

JCheckBox chkOption1 = new JCheckBox("Option 1");
JCheckBox chkOption2 = new JCheckBox("Option 2");
JCheckBox chkOption3 = new JCheckBox("Option 3");

jPanel.add(chkOption1);
jPanel.add(chkOption2);
jPanel.add(chkOption3);
```



JRadioButton, ButtonGroup

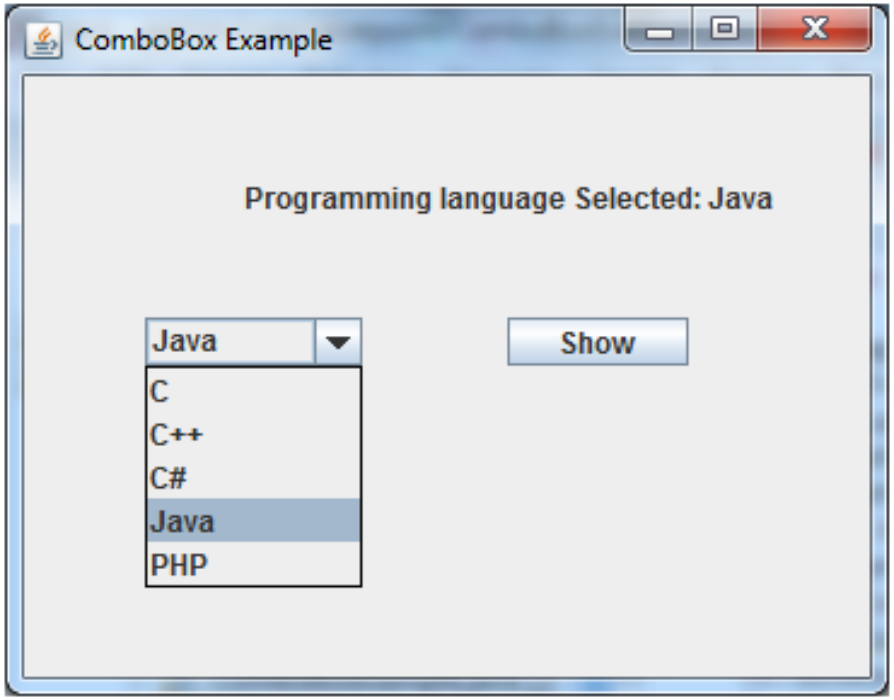
- `JRadioButton` is used to create select items. Commonly used attributes:
  - Text: Attached label
  - Selected: Status
  - By convention, names should start with `rdo`
- `ButtonGroup`: the group to which the radios belong. In each group you can only select one radio.

JRadioButton, ButtonGroup

```
JPanel jPanel = new JPanel();
jPanel.setLayout(new GridLayout(3, 1));
JLabel lblSelect = new JLabel("Select your option:");
jPanel.add(lblSelect);

JRadioButton rdoMale = new JRadioButton("Male");
JRadioButton rdoFemale = new JRadioButton("Female");

ButtonGroup genders = new ButtonGroup();
genders.add(rdoMale);
genders.add(rdoFemale);
```

JComboBox

- `JComboBox` is used to create a drop-down list and only allows selecting *one* option
  - Model: contains a list of data
  - SelectedIndex: selected item location
  - SelectedItem: selected item data
  - Name should start with `cbo` or `cb`



JComboBox

```
JPanel jPanel = new JPanel();
JLabel lblSelect = new JLabel("Select your option:");
jPanel.add(lblSelect);

String[] languages = {"C", "C++", "C#", "Java", "PHP"};
JComboBox<String> cbLanguages = new JComboBox<>(languages);
```

JTable, DefaultTableModel

- `JTable` is used to represent tabular data
  - Model: table data
  - SelectionMode: mode for selecting rows
  - RowHeight: height of each row
  - Name should start with `tbl` (e.g. `tblBooks`)

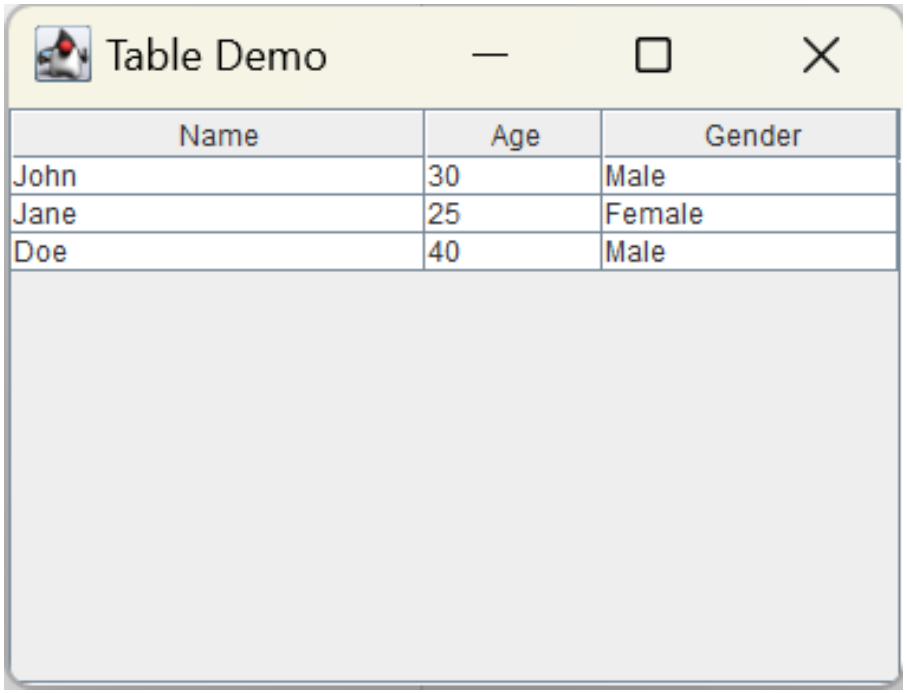
ISBN	Title	Author	Genre	Publication ...	Copies Avail...
JR-06-1997	Harry Potter	J.K. Rowling	FANTASY	1997	5
HP-03-1992	Pride and Prejudice	Harry Porter	MYSTERY	1992	2
DC-01-1992	abc hsd	dads cbs	FICTION	1992	2
TF-04-1882	abd rued	thidy fdkr	ROMANCE	1882	2
W-03-1992	Harry Potter2	Willam	MYSTERY	1992	2

JTable, DefaultTableModel

```java
// create a DefaultTableModel
DefaultTableModel model = new DefaultTableModel();
model.addColumn("Name");
model.addColumn("Age");
model.addColumn("Gender");

// add some data
model.addRow(new Object[]{"John", 30, "Male"});
model.addRow(new Object[]{"Jane", 25, "Female"});
model.addRow(new Object[]{"Doe", 40, "Male"});
```

JTable, DefaultTableModel

```
// create a JTable with a DefaultTableModel
JTable tblCustomers = new JTable(model);

// set column widths
tblCustomers.getColumnModel()
        .getColumn(0)
        .setPreferredWidth(150);
tblCustomers.getColumnModel()
        .getColumn(1)
        .setPreferredWidth(50);
tblCustomers.getColumnModel()
        .getColumn(2)
        .setPreferredWidth(100);

// add table to JScrollPane
JScrollPane scrollPane = new JScrollPane(tblCustomers);

// add scroll pane to window
frame.add(scrollPane, BorderLayout.CENTER);
```



Summary

- GUI Introduction
- Basic components of GUI
  - Container
  - JLabel
  - JTextField, JTextArea and JPasswordField
  - Jbutton
  - JCheckBox
  - JRadioButton, ButtonGroup
  - JComboBox
  - JTable, DefaultTableModel